Floating-point numbers

CSE 101 Section 03
Introduction to Algorithms and Computing
Fall 2023
© Royal Military College of Canada

```python
a = 0.1
b = 0.2
c = 0.3

# Ideally, (a + b) should equal c
result = a + b

print("a + b = ", result)
print("c = ", c)
print("a + b == c :", result == c)
```

```python
a = 0.1
sum = 0.0

# Add 0.1 to the sum 10 times
for i in range(10):
    sum += a

print("Sum = ", sum)
print("Expected sum = 1.0")
print("Sum == 1.0 :", sum == 1.0)
```

Why is this happening?

To understand why, we need to look at how floating-point numbers are converted to binary before they are stored on computer.

Floating-Point in Binary

In decimal the number `56.482` translates as

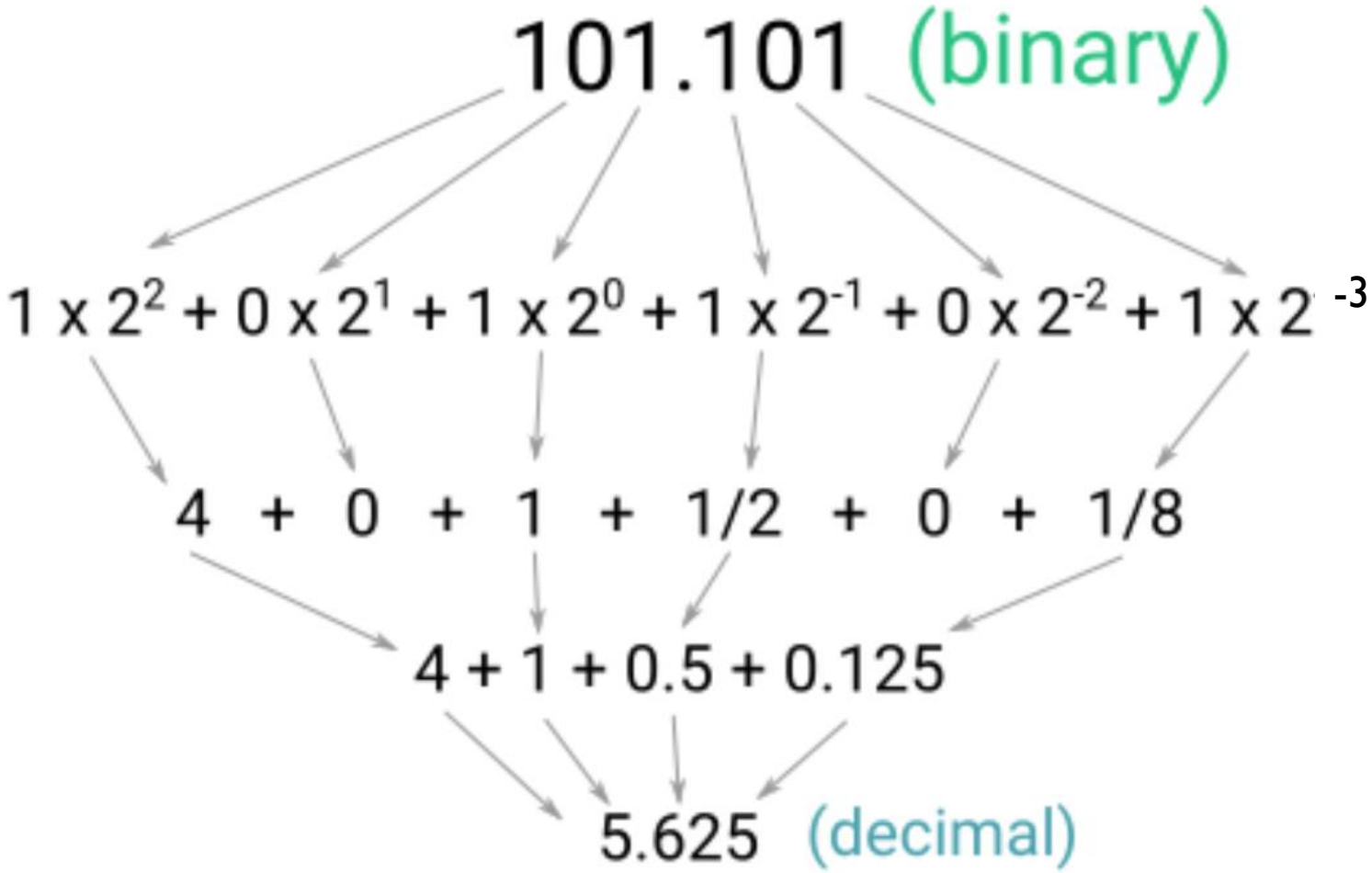
$5 * 10^1$	50
$6 * 10^0$	6
$4 * 10^{-1}$	4/10
$8 * 10^{-2}$	8/100
$2 * 10^{-3}$	2/1000

In binary it is the same process however we use powers of `2` instead.

So in binary the number `101.101` translates as

$1 * 2^2$	4
$0 * 2^1$	0
$1 * 2^0$	1
$1 * 2^{-1}$	1/2
$0 * 2^{-2}$	0
$1 * 2^{-3}$	1/8

Floating-Point in Binary



The binary point

- So far, we have represented our binary fractions with the use of a binary point.
- This is fine when we are working with things normally but within a computer this is not feasible as it can only work with `0`'s and `1`'s.
  - We get around this by agreeing where the binary point should be.

- So, for instance, if we are working with `8` bit numbers, it may be agreed that the binary point will be placed between the `4`th and `5`th bits.

`01101001` is then assumed to actually represent `0110.1001`

Converting a decimal fraction to binary

- The easiest approach is a method where we repeatedly multiply the fraction by `2` and recording whether the digit to the left of the decimal point is a `0` or `1` (i.e., if the result is greater than `1`), then discarding the `1` if it is. Once you are done you read the value from top to bottom.

`0.125`

0.250	0
0.500	0
1.000	1

`0.001`

Converting a decimal fraction to binary

- Another example:

0.16

0.32	0
0.64	0
1.28	1
0.56	0
1.12	1
0.24	0
0.48	0
0.96	0
1.92	1
1.84	1
1.68	1
…	…

0.00101000111…

Converting a decimal fraction to binary

- A decimal number can be perfectly represented in binary <u>only if</u> it is a sum of powers of 2.

- In binary, 0.5 is represented as 0.1 because: $0.5 = \frac{1}{2^1}$
- Same as 0.75: $0.75 = \frac{1}{2^1} + \frac{1}{2^2}$
- But not 0.16: <u>cannot</u> be represented with a finite number of bits.

Now how is this stored?

Floating Point

- What we have looked at previously is what is called fixed point binary fractions, convenient but….
- As soon as the number we want to represent is very large or very small, we find that we need a very large number of bits to represent them.

- To get around this we use a method of representing numbers called **floating point.**
- Floating point is quite similar to scientific notation as a means of representing numbers.

You are familiar with Floating-Point Number Anatomy

$$-3.154 * 10^{5}$$

The **sign** (negative here)
The **mantissa** (= `3.154` here)
The **exponent** (= `5` here)

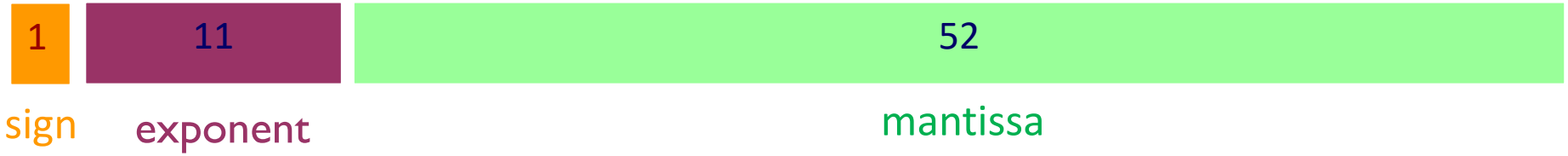
Now the same in Binary

- We may do the same in binary and this forms the foundation of our floating point number.

- Python uses what is called the IEEE 754 Standard, which tells us exactly how to represent floating point numbers.
  - The standard specifies the number of bits used for each section (exponent, mantissa and sign).
  - and the order in which they are represented

IEEE 754: Number of bits and Order

- Uses `64` bits and has the following layout.
  - **1 bit** for the sign of the number. `0` means positive and `1` means negative.
  - **11 bits** for the exponent.
  - **52 bits** for the mantissa.



Back to 0.16

- **Step 1: Convert to Binary (already done)**

$$0.00101000111\ldots$$

- **Step 2: Normalize the Binary Number**

$$1.01000111\ldots * 2^{-4}$$

Back to 0.16 = `1.01000111...*` 2^{-4}

- **Step 3: Encode According to IEEE 754**
- Now we need to place this number into the IEEE 754 format for a float, which has a sign bit, an 11-bit exponent, and a 52-bit fraction (mantissa).

  - **Sign bit**: This is easy for positive numbers; the sign bit is 0.
  - **Exponent**: The exponent is converted to binary and stored here.
  - **Mantissa**: This is the fraction part, with the leading 1 removed (since it is implied), which is `01000111`.... <u>Only the first 52 bits</u> of the fraction are kept
    ➔ This is where precision is lost. The number stored when converted back to decimal is very close to 0.16 but is not exactly equal to 0.16.

Limited Precision

Therefore, if more than 52 bits are needed for a given number, precision is lost…

Rounding Errors

- Due to limited precision, some floating-point arithmetic operations can introduce rounding errors.

- This is not a flaw of Python but a fundamental property of floating-point representation.

- Recall

```
a = 0.1
sum = 0.0
for i in range(10):
    sum += a
print("Sum = ", sum)
print("Expected sum = 1.0")
print("Sum == 1.0 :", sum == 1.0)
```

Example: What is the problem here?

```
a = 0
while a != 1:
    print(f'{a:.20f}')
    a += 0.1
```

Real world examples of rounding errors

The Patriot missile defense system

- Was used during the Gulf War and was also rendered ineffective due to rounding error.
- The system used an integer timing register which was incremented at intervals of `0.1`s.
- The binary approximation of `0.1` was used

$$0.0001100110011001100110011_2 = 209715/2097152$$

Real world examples of rounding errors

The Patriot missile defense system

After 100 hours (3.6 x 106 ticks*), an error of

$$\left(\frac{1}{10} - \frac{209715}{2097152}\right)(3.6 \times 10^6) = \frac{5625}{16834} \approx 0.3433\ seconds$$

had accumulated. This discrepancy caused the Patriot system to continuously recycle itself instead of targeting properly. As a result, an Iraqi Scud missile could not be targeted and was allowed to detonate on a barracks, killing 28 people.

Next…

Von Neumann architecture